SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct
I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud.
   Discuss how cloud storage solutions ensure durability and data redundancy.
   How can encryption and access control protect sensitive backup data?
   Explain the cost advantages compared to traditional storage systems.
   Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
   Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

Question 1: Auto-Scaling and Load Balancing for Seasonal Traffic Spikes

1.1 The Problem

An online retailer faces highly unpredictable traffic during seasonal sales such as Diwali, Black Friday or end-of-season offers. On an ordinary day the platform may serve a few thousand concurrent users, but within minutes of a flash sale starting this can jump by ten to fifty times. A traditional on-premises setup must be sized for the worst-case peak, which means expensive servers sit idle for most of the year. If the company sizes for average load instead, the website slows down or crashes exactly when revenue potential is highest. Cloud computing solves this dilemma through elasticity: resources are added when demand rises and released when demand falls, and traffic is spread intelligently across them.

1.2 How Auto-Scaling Works

Auto-scaling is a cloud service that automatically adjusts the number of compute instances (virtual machines or containers) in a group according to rules defined by the operator. Examples include Amazon EC2 Auto Scaling, Azure Virtual Machine Scale Sets and Google Compute Engine Managed Instance Groups. The main policy types are:

- **Dynamic (target tracking) scaling:** the group keeps a metric, such as average CPU utilisation at 60 percent or requests per instance, near a target value. When the metric rises above the target, new instances are launched; when it falls, surplus instances are terminated.
- **Scheduled scaling:** capacity is raised at a known time, for example 30 minutes before an announced midnight sale, so that servers are warm before customers arrive.
- **Predictive scaling:** machine-learning models study historical traffic patterns and provision capacity ahead of forecast demand.
- **Horizontal versus vertical scaling:** horizontal scaling (adding more instances) is preferred for web workloads because it has no practical upper limit and avoids a single point of failure, whereas vertical scaling (a larger instance) is bounded by hardware size and usually needs a restart.

Each new instance is created from a pre-configured machine image or container image, so it joins the fleet already containing the application code and configuration. Minimum and maximum limits protect the business from both under-provisioning and runaway cost.

1.3 How Load Balancing Works

A load balancer is the single entry point that receives all incoming requests and distributes them across the healthy back-end servers. Application load balancers (Layer 7) can route by URL path, host name or HTTP header, for example sending /checkout to payment-optimised servers and /images to a separate pool. Network load balancers (Layer 4) handle very high connection volumes with extremely low latency. Common distribution algorithms include round robin, least connections and weighted routing. Crucially, the load balancer performs regular health checks and automatically stops sending traffic to any instance that fails to respond, so one faulty server never causes an outage for customers.

Auto-scaling and load balancing are complementary. Auto-scaling decides how many servers exist, and the load balancer decides which server receives each request. When auto-scaling launches a new instance, it is automatically registered with the load balancer, and when an instance is removed, the load balancer first drains its existing connections so that no customer session is dropped.

1.4 Reducing Downtime and Improving User Experience

- **No single point of failure:** servers are spread across multiple Availability Zones. If one data centre fails, the load balancer routes traffic to the remaining zones and auto-scaling replaces the lost capacity.
- **Self-healing:** instances that fail health checks are terminated and replaced automatically without human intervention, which shortens recovery time from hours to minutes.
- **Consistent response time:** because capacity grows with demand, page load and checkout latency remain low even under peak load, which lowers cart abandonment and increases conversion.
- **Graceful updates:** new versions can be rolled out instance by instance (rolling or blue/green deployment) while the load balancer keeps serving users, so releases do not require downtime.
- **Cost efficiency:** the retailer pays only for the instances that are actually running, and capacity shrinks back after the sale ends.

1.5 Role of Monitoring Tools

Monitoring is the feedback loop that makes the whole system work. Services such as Amazon CloudWatch, Azure Monitor and Google Cloud Operations collect metrics (CPU, memory, request count, latency, error rate, queue depth), logs and traces. Auto-scaling policies are driven directly by these metrics, so poor metric choice leads to poor scaling. Monitoring also provides:

- Alarms and notifications (email, SMS, chat) when error rates or latency cross a threshold, so engineers can intervene early.
- Dashboards that show real-time business and technical indicators during the sale, such as orders per minute and database connections.
- Historical data for capacity planning, post-sale analysis and tuning of scaling thresholds and cooldown periods.
- Distributed tracing to locate bottlenecks, for example a slow payment gateway call, rather than blindly adding more servers.

1.6 Real-World E-Commerce Scenario

Consider an Indian e-commerce company running a “Big Sale Day” that opens at midnight. The architecture places the web and API tier in an auto-scaling group across three Availability Zones behind an application load balancer. Product images and static files are served from object storage through a CDN, and product and order data are kept in a managed database with read replicas and a caching layer. Before the sale, scheduled scaling raises the group from 10 to 60 instances. At midnight traffic surges and CPU utilisation climbs above 60 percent, so target-tracking policies add instances up to a ceiling of 300. Monitoring alarms notify the operations team when the checkout latency exceeds two seconds, and they respond by increasing database read replicas. In the early morning, when traffic falls, instances are terminated automatically. The result is a smooth shopping experience, no outage, and an infrastructure bill that reflects actual usage rather than permanent peak capacity.

Question 2: Secure and Cost-Effective Cloud Backup Storage

2.1 Durability and Data Redundancy

Durability is the probability that stored data remains intact and is not lost over a period of time, while availability is the probability that the data can be accessed when requested. Cloud object storage services such as Amazon S3, Azure Blob Storage and Google Cloud Storage are designed for extremely high durability, commonly quoted as 99.999999999 percent (eleven nines) per year for standard classes. Providers achieve this through several mechanisms:

- **Replication:** every object is automatically stored as multiple copies on different disks, servers and, in multi-zone classes, in physically separate data centres (Availability Zones) within a region.
- **Erasure coding:** data is split into fragments with parity information so that the object can be reconstructed even if several fragments are lost, using less raw capacity than simple copying.
- **Integrity checking:** checksums are calculated on upload and verified continuously; corrupted copies are detected and automatically repaired from healthy copies.
- **Geo-redundancy:** cross-region replication keeps a copy in a distant region to protect against regional disasters such as floods, power-grid failures or large-scale outages.
- **Versioning:** previous versions of an object are retained, which protects against accidental overwrite or deletion.

Organisations select a redundancy level according to their requirements: locally redundant storage is the cheapest, zone-redundant storage survives the loss of a data centre, and geo-redundant storage survives the loss of an entire region.

2.2 Protecting Backups with Encryption and Access Control

Backups often contain the most sensitive information an organisation owns, including customer records, financial data and source code, so they are an attractive target for attackers. Cloud platforms provide layered protection:

- **Encryption in transit:** data travels to the cloud over TLS/HTTPS or a private connection (VPN or dedicated link) so that it cannot be read or altered on the network.
- **Encryption at rest:** data is encrypted with strong algorithms such as AES-256 before it is written to disk. The provider can manage the keys, the customer can manage keys through a key-management service (AWS KMS, Azure Key Vault, Google Cloud KMS), or the customer can hold keys entirely on-premises (client-side encryption) for maximum control.
- **Key management:** keys are rotated regularly, stored in hardware security modules and their use is logged, so that stealing a disk does not expose the data.
- **Identity and access management (IAM):** access follows the principle of least privilege. Only specific users and roles may write or restore backups, permissions are granted through policies, and multi-factor authentication is enforced for administrators.
- **Immutability:** features such as S3 Object Lock or Azure immutable blob storage use a write-once-read-many (WORM) model so that backups cannot be deleted or modified for a fixed retention period, even by a compromised administrator account. This is a major defence against ransomware.
- **Network and audit controls:** private endpoints, bucket policies that block public access, and audit logs of every access request.

2.3 Cost Advantages over Traditional Storage

Traditional backup requires buying tape libraries, disk arrays and secondary-site facilities in advance, then paying for power, cooling, floor space, maintenance staff and periodic hardware replacement. The cloud shifts spending from capital expenditure to operating expenditure and removes most of these overheads.

Aspect	Traditional Storage	Cloud Storage
Upfront cost	High capital investment in hardware and facilities	None; pay-as-you-go
Capacity planning	Must be over-provisioned for future growth	Virtually unlimited; scale on demand
Maintenance	In-house staff, hardware refresh every 3-5 years	Managed by the provider
Redundancy	Needs a second site that is paid for in full	Built in at a small additional price
Cost optimisation	Little flexibility	Lifecycle policies move old data to cheaper tiers (for example archive classes such as S3 Glacier)

Tiered storage is especially valuable for backups. Recent backups stay in a standard class for quick restore, older ones move automatically to infrequent-access and archive tiers costing a small fraction of the price, and expired backups are deleted by lifecycle rules. Compression and deduplication reduce the stored volume further.

2.4 Disaster Recovery Use Case

Suppose a hospital group stores patient records on servers in its main data centre. It performs nightly backups encrypted with customer-managed keys to a cloud bucket with versioning, object lock and cross-region replication to a second region. One night a ransomware attack encrypts the production servers and the backup server on the local network. Because the cloud copies are immutable and protected by separate credentials, the attackers cannot alter them. The recovery team provisions virtual machines in the cloud from pre-built images, restores the most recent clean backup and redirects users to the temporary environment. The recovery point objective (RPO), the amount of data lost, is limited to the last backup interval, and the recovery time objective (RTO), the time to resume service, is measured in hours rather than days. Regular test restores and documented runbooks make sure this plan works when it is needed.

Question 3: Global Deployment Using CDNs and Cloud Availability

3.1 Why Latency Matters

A mobile application company launching worldwide must serve users in India, Europe, North America and elsewhere with similar responsiveness. Network latency is limited by the physical distance data must travel and by the number of network hops. A user in Chennai fetching content from a server in Virginia will experience hundreds of milliseconds of round-trip delay for every request, which results in slow screens, buffering videos and laggy interactions. Slow applications are uninstalled quickly, so performance is directly related to retention and revenue.

3.2 How a Content Delivery Network Improves Performance

A content delivery network (CDN) is a geographically distributed network of servers that cache content close to users. Examples are Amazon CloudFront, Azure CDN, Google Cloud CDN, Cloudflare and Akamai. When a user requests a file, DNS or anycast routing directs the request to the nearest edge location. If the edge server already holds the object (a cache hit), it is delivered immediately. If not (a cache miss), the edge fetches it from the origin server once, stores it according to its time-to-live setting, and serves later users from the cache. The key benefits are:

- **Reduced latency:** content travels a short distance, so round-trip time falls from hundreds to tens of milliseconds.
- **Reduced origin load:** most requests never reach the origin, which saves compute, bandwidth and cost and allows the origin to handle dynamic requests.
- **Optimised delivery:** CDNs compress content, use modern protocols (HTTP/2, HTTP/3, QUIC), keep persistent connections to the origin and terminate TLS close to the user, which speeds up connection setup.
- **Better reliability:** if one edge location or network path is congested or fails, traffic is rerouted to another.
- **Security:** CDNs absorb distributed denial-of-service (DDoS) attacks at the edge and provide web-application firewalls and bot protection.

3.3 How Edge Servers Enhance User Experience

Edge servers sit in points of presence (PoPs) inside or near internet service provider networks in major cities. They store static assets such as images, app updates, JavaScript and video segments, and increasingly they also run code. Edge computing features such as Lambda@Edge, Cloudflare Workers and Azure Front Door rules allow authentication checks, A/B testing, URL rewriting, personalisation and API response caching to be performed at the edge without a trip to the origin. For a mobile app this means that launch screens, configuration files and media load quickly regardless of the user's region, large downloads are resumed from nearby servers, and users on congested mobile networks have fewer hops to travel.

3.4 Global Availability Supported by Cloud Providers

CDNs handle content delivery, but the application back end must also be globally available. Cloud providers support this through:

- **Regions and Availability Zones:** dozens of regions around the world, each containing multiple isolated zones, allow the application to be deployed close to users and to survive the failure of a zone.
- **Global traffic management:** services such as Amazon Route 53, Azure Traffic Manager and Google Cloud Load Balancing use latency-based, geolocation or failover routing to send each user to the closest healthy region.
- **Multi-region data replication:** globally distributed databases such as DynamoDB Global Tables, Cosmos DB or Cloud Spanner replicate data between regions so that users read and write locally.
- **Private global backbone:** traffic between provider regions travels on the provider's own high-speed network instead of the unpredictable public internet.
- **Elasticity per region:** auto-scaling operates independently in each region to follow local demand patterns across time zones.

3.5 Application to Streaming and Gaming

Video streaming. A streaming service encodes each video into many bitrates and splits it into small segments of a few seconds. These segments are cached on edge servers (Netflix places its Open Connect appliances inside ISP networks, and live sports platforms distribute millions of simultaneous viewers through multi-CDN setups). The player measures network speed and requests the appropriate quality using adaptive bitrate streaming, so users in different regions get fast start-up, minimal buffering and smooth playback. During a popular live match, the origin only serves each segment to the CDN once, while the edge servers serve it to millions of viewers.

Online gaming. Games cannot cache gameplay state because it changes constantly, but a CDN still delivers game patches, textures and assets quickly. For real-time play, cloud providers run game servers in regions close to players, and matchmaking services direct players to the lowest-latency region. Edge locations and the provider's backbone reduce jitter and packet loss. A player in Mumbai connects to a server in a nearby Indian region rather than one in Europe, so ping is low and the game feels responsive. Together, these techniques give a consistent experience to a worldwide user base.

Question 4: Compliance, Security and Data Sovereignty for a Financial Institution

4.1 Regulatory Compliance and Auditing Support

A bank or payments company operates under strict regulations governing confidentiality, integrity, availability and record keeping. Examples include PCI DSS for card data, ISO/IEC 27001 for information security management, SOC 1 and SOC 2 for service-organisation controls, GDPR for personal data of European residents, and country-specific regulations such as the Reserve Bank of India's guidelines on outsourcing, cyber security and payment-data storage. Cloud providers support compliance through the following:

- **Shared responsibility model:** the provider is responsible for security of the cloud (physical data centres, hardware, hypervisor and core services), while the customer is responsible for security in the cloud (identity management, configuration, data classification, encryption choices and application security). Understanding this division prevents gaps.
- **Third-party certifications and reports:** providers undergo independent audits and publish attestations through portals such as AWS Artifact, Microsoft Service Trust Portal and Google Compliance Reports Manager, which the institution can present to its own auditors and regulators.
- **Compliance tooling:** services such as AWS Config, Azure Policy and Google Security Command Center continuously check resources against rule sets and benchmarks (for example CIS) and flag or automatically remediate violations.
- **Audit trails:** every API call and configuration change is recorded, so auditors can see who did what and when.
- **Contractual commitments:** data processing agreements, right-to-audit clauses and breach-notification commitments in the service contract.

4.2 Role of Logging, Monitoring and Encryption

Logging creates a tamper-resistant record of activity. Services such as AWS CloudTrail, Azure Activity Log and Cloud Audit Logs capture administrative actions, while application, database and network flow logs capture user and system behaviour. Logs are stored in write-protected, encrypted

storage for the retention period required by regulation and are essential for forensic investigation and evidence during an audit.

Monitoring analyses logs and metrics in real time. Security information and event management (SIEM) tools and threat-detection services (for example Amazon GuardDuty, Microsoft Defender for Cloud) detect suspicious behaviour such as logins from unusual locations, privilege escalation, large data transfers or changes to security groups. Automated alerts and response workflows allow incidents to be contained quickly, which meets regulatory obligations to detect and report breaches promptly.

Encryption protects confidentiality. Data at rest in databases, disks and object storage is encrypted with AES-256, and data in transit is protected with TLS 1.2 or higher. Keys are managed in hardware security modules; customer-managed keys allow the bank to control rotation and revoke access, and may be held in a dedicated HSM or an external key manager. Sensitive fields such as card numbers can additionally be tokenised or encrypted at the application level so that even database administrators cannot see them.

Other essential controls include strong identity management (multi-factor authentication, role-based access, just-in-time privileged access), network segmentation with private subnets and firewalls, and regular vulnerability scanning and penetration testing.

4.3 Ensuring Data Sovereignty

Data sovereignty means that data is subject to the laws of the country where it is stored or processed, and some regulations require certain data to remain within national borders. The institution can ensure compliance by:

- Selecting cloud regions inside the required country or jurisdiction, and choosing services that guarantee data residency within that region.
- Applying policy guardrails (for example organisation policies or service control policies) that deny the creation of resources in any non-approved region.
- Disabling cross-region replication and backups to other geographies, or restricting them to approved locations.
- Keeping encryption keys in the same jurisdiction, or holding them on-premises, so that foreign authorities or the provider cannot access plaintext data.
- Using sovereign or dedicated cloud offerings, or private and hybrid models, where local regulation demands them.
- Reviewing sub-processors and support access arrangements in the contract, and regularly auditing data flows and storage locations.

4.4 Example: Sensitive Financial Transactions

Consider a payments company processing UPI and card transactions for Indian customers. Regulations require that payment data be stored only in India. The company deploys its application in an Indian cloud region and blocks other regions through policy. When a customer initiates a payment, the request arrives over TLS at a web application firewall and load balancer, then passes to microservices in private subnets. The card number is tokenised, the transaction record is written to an encrypted database whose key is held in a customer-managed key service within India, and every access to the data is logged to immutable storage. Monitoring tools raise alerts for abnormal patterns such as many failed authorisations from one device, and fraud-detection models run in the same region. When regulators or auditors ask for evidence, the company presents the provider’s

certifications, its own configuration compliance reports and the audit logs, demonstrating that the data stayed in the country, was encrypted, and was accessed only by authorised users.

Question 5: Containerization, Orchestration and Portability

5.1 Containers versus Virtual Machines

A virtual machine (VM) emulates a complete computer. A hypervisor (Type 1 such as KVM, Hyper-V and ESXi, or Type 2) shares the physical hardware among several VMs, and each VM runs its own full guest operating system with its own kernel, libraries and application. A container, by contrast, packages an application with its dependencies but shares the host operating-system kernel. Isolation is provided by Linux kernel features: namespaces isolate process views, networks and file systems, while control groups (cgroups) limit CPU, memory and I/O. Container engines such as Docker and containerd run containers from lightweight, layered images.

Parameter	**Virtual Machine**	**Container**
Isolation level	Hardware-level; separate guest OS and kernel	Process-level; shared host kernel
Size	Gigabytes (includes full OS)	Megabytes (application and libraries only)
Start-up time	Minutes or tens of seconds	Milliseconds to seconds
Resource usage	Higher; each VM reserves memory and CPU for its OS	Lower; no duplicated OS, so many more instances per host
Performance	Some overhead from hypervisor and guest OS	Near-native performance
Security	Stronger isolation boundary; a kernel exploit affects only that VM	Weaker boundary because the kernel is shared; needs hardening (seccomp, least privilege, image scanning)
Portability	Heavy images tied to hypervisor format	Same image runs anywhere with a container runtime
Typical use	Running different operating systems, strong multi-tenant isolation, legacy applications	Microservices, CI/CD pipelines, scalable cloud-native applications

In practice, performance and density favour containers: a server that hosts ten VMs may run hundreds of containers because each one avoids a duplicate operating system. Security considerations, however, mean that many cloud providers run containers inside lightweight VMs or on dedicated nodes, and the two technologies are frequently combined.

5.2 How Orchestration with Kubernetes Simplifies Deployment and Scaling

Running a few containers by hand is simple, but production systems may involve hundreds of containers across many hosts. An orchestrator automates their deployment, networking, scaling and recovery. Kubernetes, originally developed by Google and now maintained by the Cloud Native Computing Foundation, is the dominant orchestrator and is available as managed services such as Amazon EKS, Azure AKS and Google GKE. Its main concepts and features are:

- **Cluster architecture:** a control plane (API server, scheduler, controller manager, etcd) manages worker nodes, each running a kubelet and a container runtime. The smallest deployable unit is a Pod, which holds one or more containers.
- **Declarative deployment:** the operator describes the desired state in YAML (for example, "run five replicas of version 2.1 of the cart service"), and Kubernetes continuously works to make the actual state match.
- **Scheduling:** containers are placed on nodes with enough resources according to requests, limits and affinity rules.
- **Scaling:** the Horizontal Pod Autoscaler adjusts the number of Pods according to CPU or custom metrics, and the cluster autoscaler adds or removes nodes.
- **Self-healing:** failed containers are restarted, unresponsive Pods are replaced and workloads are rescheduled when a node fails.
- **Service discovery and load balancing:** Services and Ingress provide stable network names and distribute traffic across Pods.
- **Rolling updates and rollbacks:** new versions replace old ones gradually, and a faulty release can be reverted with one command.
- **Configuration and secrets management:** ConfigMaps and Secrets separate configuration from images, and persistent volumes provide storage for stateful services.

5.3 Portability across Cloud Environments

A container image contains everything the application needs: code, runtime, libraries and configuration defaults. This solves the "it works on my machine" problem, because the same image behaves identically on a developer laptop, a test server, a private data centre and any public cloud. The Open Container Initiative (OCI) standards make images and runtimes interchangeable between tools. Kubernetes adds a common API layer, so manifests and Helm charts written for one provider can be applied on another with few changes. The benefits are:

- **Avoiding vendor lock-in:** the organisation can move workloads between AWS, Azure, Google Cloud or on-premises with limited rework.
- **Hybrid and multi-cloud strategies:** workloads can be placed according to cost, regional availability or compliance, and failover across providers is possible.
- **Consistent development and operations:** the same images and pipeline are used for every environment, which reduces configuration drift and release errors.
- **Faster migration and disaster recovery:** images stored in a registry can be redeployed rapidly in another location.

Portability is not absolute: dependencies on provider-specific managed databases, load balancers and identity services can still create lock-in, so teams should use open standards and abstract such dependencies where possible.

5.4 Example: Microservices-Based Application

An online food-delivery application illustrates the approach. Instead of one large monolith, it is divided into independent microservices: a User Service for accounts and authentication, a Restaurant Catalogue Service for menus, a Cart and Order Service, a Payment Service, a Delivery Tracking Service and a Notification Service. Each service has its own codebase, container image and database, and communicates with the others through REST APIs or a message queue. An API gateway receives all client requests and routes them to the correct service.

On Kubernetes, each service is deployed as a separate Deployment with its own scaling rules. At dinner time the Order and Delivery Tracking services scale from three to thirty Pods while the User Service stays small. If the Payment Service has a bug, only that service is rolled back, and the rest of the application continues to work. Teams can use different programming languages for different services and release updates several times a day without redeploying the whole system. The same manifests allow the application to run on a managed Kubernetes service in any cloud, demonstrating the speed, resilience and portability that containerization and orchestration provide.

Conclusion

The five scenarios show how the core characteristics of cloud computing (elasticity, resource pooling, broad network access, measured service and on-demand self-service) translate into practical solutions. Auto-scaling and load balancing give resilience under variable load, cloud storage provides durable and affordable backups, CDNs and global infrastructure bring applications close to users, compliance and security controls make regulated workloads possible, and containers with Kubernetes deliver efficient, portable deployment. Correct application of these concepts depends on understanding the underlying virtualization and isolation technologies, and on monitoring, security and cost controls that keep the environment reliable.